

Q1. Create a **user-defined exception** for insufficient bank balance during withdrawal.

Lab_09 > Q1.java > Q1 > main(String[])

```
1 package Lab_09;
2 class InsufficientBalanceException extends Exception {
3     public InsufficientBalanceException(String message) {
4         super(message);
5     }
6 }
7
8 class BankAccount {
9     double balance = 5000;
10
11     void withdraw(double amount) throws InsufficientBalanceException {
12         if (amount > balance) {
13             throw new InsufficientBalanceException(message: "Insufficient Balance!");
14         } else {
15             balance -= amount;
16             System.out.println(x: "Withdrawal Successful");
17             System.out.println("Remaining Balance: " + balance);
18         }
19     }
20 }
21
22 public class Q1 {
23     Run | Debug
24     public static void main(String[] args) {
25         BankAccount b = new BankAccount();
26
27         try {
28             b.withdraw(amount: 6000);
29         } catch (InsufficientBalanceException e) {
30             System.out.println(e.getMessage());
31         }
32     }
33 }
```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS

- gauravmathur@Annus-Mac LabJava % javac Lab_09/Q1.java
- gauravmathur@Annus-Mac LabJava % java Lab_09.Q1
Insufficient Balance!
- gauravmathur@Annus-Mac LabJava %

Q2. Write a program to demonstrate the use of **multiple catch blocks** for handling different types of exceptions.

Lab_09 > Q2.java > { } Lab_09

```
1 package Lab_09;
2 public class Q2 {
3     public static void main(String[] args) {
4
5         try {
6             int a = 10 / 0;
7
8             int arr[] = new int[5];
9             arr[10] = 50;
10
11             String s = null;
12             System.out.println(s.length());
13
14         } catch (ArithmeticException e) {
15             System.out.println(x: "Arithmetic Exception Occurred");
16
17         } catch (ArrayIndexOutOfBoundsException e) {
18             System.out.println(x: "Array Index Exception Occurred");
19
20         } catch (NullPointerException e) {
21             System.out.println(x: "Null Pointer Exception Occurred");
22
23         } finally {
24             System.out.println(x: "Program Finished");
25         }
26     }
27 }
```

OUTPUT:—

```
● gauravmathur@Annus-Mac LabJava % javac Lab_09/Q2.java
● gauravmathur@Annus-Mac LabJava % java Lab_09.Q2
Arithmetic Exception Occurred
Program Finished
gauravmathur@Annus-Mac LabJava %
```

Q3. Write a program to demonstrate the use of **throws keyword** with checked exceptions to postpone the handling of checked exception.

Lab_09 >  Q3.java > ...

```
1  package Lab_09;
2  import java.io.*;
3
4  public class Q3 {
5
6      static void checkFile() throws IOException {
7          FileReader file = new FileReader(fileName: "abc.txt");
8          BufferedReader br = new BufferedReader(file);
9
10         System.out.println(br.readLine());
11
12         br.close();
13     }
14
15     Run | Debug
16     public static void main(String[] args) {
17         try {
18             checkFile();
19         } catch (IOException e) {
20             System.out.println(x: "File Not Found");
21         }
22     }
```

OUTPUT:—

```
Program Finished
● gauravmathur@Annus-Mac LabJava % javac Lab_09/Q3.java
● gauravmathur@Annus-Mac LabJava % java Lab_09.Q3
File Not Found
❖ gauravmathur@Annus-Mac LabJava %
```

Q4. Write a program and create multiple threads in it and then display the **name, priority, and state** of the currently executing thread. Also, try to assign custom names and priority to the threads

```
Lab_09 > Q4.java > ...
1  package Lab_09;
2  class MyThread extends Thread {
3
4      public MyThread(String name) {
5          super(name);
6      }
7
8      public void run() {
9          System.out.println("Thread Name: " + Thread.currentThread().getName());
10         System.out.println("Priority: " + Thread.currentThread().getPriority());
11         System.out.println("State: " + Thread.currentThread().getState());
12
13         for (int i = 1; i <= 5; i++) {
14             System.out.println(getName() + " : " + i);
15         }
16     }
17 }
18
19 public class Q4 {
20     Run | Debug
21     public static void main(String[] args) {
22
23         MyThread t1 = new MyThread(name: "Thread-A");
24         MyThread t2 = new MyThread(name: "Thread-B");
25
26         t1.setPriority(Thread.MIN_PRIORITY);
27         t2.setPriority(Thread.MAX_PRIORITY);
28
29         t1.start();
30         t2.start();
31     }
32 }
```

OUTPUT:—

```
● gauravmathur@Annus-Mac LabJava % javac Lab_09/Q4.java
● gauravmathur@Annus-Mac LabJava % java Lab_09.Q4
Thread Name: Thread-B
Thread Name: Thread-A
Priority: 10
Priority: 1
State: RUNNABLE
State: RUNNABLE
Thread-B : 1
Thread-B : 2
Thread-B : 3
Thread-B : 4
Thread-B : 5
Thread-A : 1
Thread-A : 2
Thread-A : 3
Thread-A : 4
Thread-A : 5
❖ gauravmathur@Annus-Mac LabJava %
```

Q5. Write a program to demonstrate that a daemon thread terminates automatically after all user threads finish execution.

```
Lab_09 > Q5.java > Q5 > main(String[])
1  package Lab_09;
2  class DemoThread extends Thread {
3
4      public void run() {
5          if (Thread.currentThread().isDaemon()) {
6              System.out.println(x: "Daemon Thread Running");
7          } else {
8              System.out.println(x: "User Thread Running");
9          }
10
11         for (int i = 1; i <= 5; i++) {
12             System.out.println(i);
13         }
14     }
15 }
16
17 public class Q5 {
18     Run | Debug
19     public static void main(String[] args) {
20
21         DemoThread t1 = new DemoThread();
22         DemoThread t2 = new DemoThread();
23
24         t1.setDaemon(on: true);
25
26         t1.start();
27         t2.start();
28     }
29 }
```

OUTPUT:—

```
● gauravmathur@Annus-Mac LabJava % javac Lab_09/Q5.java
● gauravmathur@Annus-Mac LabJava % java Lab_09.Q5
Daemon Thread Running
1
2
User Thread Running
3
4
5
1
2
3
4
5
gauravmathur@Annus-Mac LabJava %
```

Q6. Write a program to simulate a **bank account system** where multiple threads perform deposit and withdrawal operations on the same account.

```
Lab_09 > Q6.java > Q6 > main(String[])
1 package Lab_09;
2 class BankAccountQ6 {
3     int balance = 1000;
4     synchronized void deposit(int amount) {
5         balance += amount;
6
7         System.out.println(Thread.currentThread().getName()
8             + " Deposited: " + amount);
9
10        System.out.println("Balance: " + balance);
11    }
12    synchronized void withdraw(int amount) {
13
14        if (amount <= balance) {
15
16            balance -= amount;
17
18            System.out.println(Thread.currentThread().getName()
19                + " Withdrawn: " + amount);
20
21            System.out.println("Balance: " + balance);
22        } else {
23            System.out.println("Insufficient Balance");
24        }
25    }
26 }
27
28 class DepositThread extends Thread {
29     BankAccountQ6 b;
30     DepositThread(BankAccountQ6 b) {
31         this.b = b;
32     }
33
34     public void run() {
35         b.deposit(amount: 500);
36     }
37 }
38 class WithdrawThread extends Thread {
39     BankAccountQ6 b;
40
41     WithdrawThread(BankAccountQ6 b) {
42         this.b = b;
43     }
44
45     public void run() {
46         b.withdraw(amount: 700);
47     }
48 }
49
50 public class Q6 {
51
52     Run | Debug
53     public static void main(String[] args) {
54         BankAccountQ6 b = new BankAccountQ6();
55         DepositThread t1 = new DepositThread(b);
56         WithdrawThread t2 = new WithdrawThread(b);
57
58         t1.start();
59         t2.start();
60
61     }
62 }
```

```
● gauravmathur@Annus-Mac LabJava % javac Lab_09/Q6.java
● gauravmathur@Annus-Mac LabJava % java Lab_09.Q6
Thread-0 Deposited: 500
Balance: 1500
Thread-1 Withdrawn: 700
Balance: 800
gauravmathur@Annus-Mac LabJava %
```

eadv

Q7. Write a program in which one thread is displaying the even numbers while other thread is displaying the odd numbers. If output is mixed, then apply synchronization on them so that output is not mixed.

```
Lab_09 > Q7.java > Q7 > main(String[])
1 package Lab_09;
2 class NumberDisplay {
3     synchronized void printEven() {
4         for (int i = 2; i <= 10; i += 2) {
5             System.out.println("Even: " + i);
6         }
7     }
8     synchronized void printOdd() {
9         for (int i = 1; i <= 10; i += 2) {
10            System.out.println("Odd: " + i);
11        }
12    }
13 }
14 class EvenThread extends Thread {
15     NumberDisplay n;
16     EvenThread(NumberDisplay n) {
17         this.n = n;
18     }
19     public void run() {
20         n.printEven();
21     }
22 }
23 class OddThread extends Thread {
24     NumberDisplay n;
25     OddThread(NumberDisplay n) {
26         this.n = n;
27     }
28     public void run() {
29         n.printOdd();
30     }
31 }
32 public class Q7 {
33     Run | Debug
34     public static void main(String[] args) {
35         NumberDisplay n = new NumberDisplay();
36         EvenThread t1 = new EvenThread(n);
37         OddThread t2 = new OddThread(n);
38         t1.start();
39         t2.start();
40     }
}
```

```
gauravmathur@Annus-Mac LabJava % javac Lab_09/Q7.java
gauravmathur@Annus-Mac LabJava % java Lab_09.Q7
Even: 2
Even: 4
Even: 6
Even: 8
Even: 10
Odd: 1
Odd: 3
Odd: 5
Odd: 7
Odd: 9
gauravmathur@Annus-Mac LabJava %
```

Q8. Develop a **movie theater seat booking system** using multithreading and synchronization.

```
Lab_09 > Q8.java > Q8 > main(String[])
1  package Lab_09;
2  class MovieSeatBooking {
3      int totalSeats = 5;
4      synchronized void bookSeat(int seats) {
5          System.out.println(Thread.currentThread().getName()
6              + " trying to book " + seats + " seats");
7          if (totalSeats >= seats) {
8              totalSeats -= seats;
9              System.out.println(seats + " Seats Booked Successfully");
10             System.out.println("Seats Left: " + totalSeats);
11         } else {
12             System.out.println("Seats Not Available");
13             System.out.println("Seats Left: " + totalSeats);
14         }
15     }
16 }
17 class BookingThread extends Thread {
18     MovieSeatBooking b;
19     int seats;
20     BookingThread(MovieSeatBooking b, int seats) {
21         this.b = b;
22         this.seats = seats;
23     }
24
25     public void run() {
26         b.bookSeat(seats);
27     }
28 }
29 public class Q8 {
30     public static void main(String[] args) {
31         MovieSeatBooking b = new MovieSeatBooking();
32         BookingThread t1 = new BookingThread(b, seats: 2);
33         BookingThread t2 = new BookingThread(b, seats: 3);
34         BookingThread t3 = new BookingThread(b, seats: 2);
35         t1.start();
36         t2.start();
37         t3.start();
38     }
39 }
```

OUTPUT:—

```
gauravmathur@Annus-Mac LabJava % javac Lab_09/Q8.java
gauravmathur@Annus-Mac LabJava % java Lab_09.Q8
Thread-0 trying to book 2 seats
2 Seats Booked Successfully
Seats Left: 3
Thread-2 trying to book 2 seats
2 Seats Booked Successfully
Seats Left: 1
Thread-1 trying to book 3 seats
Seats Not Available
Seats Left: 1
gauravmathur@Annus-Mac LabJava %
```